Development Individual Project Presentation Transcript

Presenter: Abdulrahman Husain Ahmed Alhashmi

Project: Python based intelligent agent for academic search and data management

Good day everyone. My name is Abdulrahman Husain Ahmed Alhashmi Thank you for joining. Today I will present the design, development, and demonstration of my Python based intelligent agent for academic search and data management. I will cover the problem, the agent model, the architecture, the scraping strategy, the data model, testing, the demo run, results, and future work. I will keep slides light. I will speak to the detail.

Slide 1 — Title and aim

Please look at the title slide. The aim is clear. Build an agent that takes an academic query, retrieves HTML inputs, parses useful records, and stores clean tables for downstream analysis. The agent reduces repetitive work during literature reviews. It turns messy pages into tidy records you can sort and analyse.

Three goals guided the work.

• Automate retrieval from HTML inputs, with room to connect to live sources.

• Structure results into a tabular form with consistent metadata.

• Support repeatability through logging, tests, and a clear README.

These goals shape the rest of the design.

Slide 2 — Problem framing

Academic search is often slow. We copy titles, authors, and years by hand. We paste URLs. We attempt to clean duplicates. Errors creep in.

I framed the problem as an agent task with explicit beliefs, desires, and intentions.

• Beliefs: what data the agent has retrieved so far.

• Desires: the user’s target query and quality thresholds.

• Intentions: the plan for scraping, parsing, filtering, and storing data.

The agent keeps state. It knows what it has done and what remains. This improves traceability and control. By modelling the workflow as an agent, I keep each step deliberate, inspectable, and testable.

Slide 3 — Conceptual foundations: agent-oriented principles

On this slide I link the design to mainstream agent thinking.

I drew on the BDI view — beliefs, desires, intentions — as set out in the multiagent systems literature.

In practice, I map these ideas to code modules and clear data flows.

• Beliefs map to the records already collected and validated.

• Desires map to the query string, the date range, and the quality checks.

• Intentions map to a concrete plan: fetch, parse, normalise, deduplicate, validate, and persist.

Autonomy sits at the workflow level. The agent decides which parser to invoke based on page type. The design supports extension. You can later add new parsers or filters without rewriting the core.

Slide 4 — Architecture overview: five modules

Here is the architecture. There are five modules, each with one main job.

1) Agent Controller. Orchestrates the workflow, passes state between modules, and logs each step.

2) Scraper. Handles HTTP requests and HTML access. It can switch to a WebDriver for dynamic pages if needed.

3) Data Processing. Converts raw parse output into a panda DataFrame, enforces schema, and carries out cleaning.

4) Storage Handler. Writes CSV and Excel outputs. Verifies row counts and file existence.

5) Logger. Captures each event with level, message, and timestamp.

The boundaries are sharp. Each module has a short public interface. This keeps reasoning clear. It also aids testing.

Slide 5 — Scraping and parsing strategy

The agent supports two modes.

• Static mode. Use Requests and Beautiful Soup. Best for predictable HTML. This is the default for the demo to ensure reproducibility.

• Dynamic mode. Use Selenium or a compatible WebDriver to render JavaScript pages when needed. Parsing focuses on article like containers. The parser extracts title, authors, year, and URL. It handles noise gracefully. If a field is missing, the parser sets a default value and flags it in the log. If the year looks invalid, the agent records None and warns. This helps when pages are messy. The agent does not stop on the first odd record. It carries on and keeps a trace.

Slide 6 — Data model and validation

The Data Processing module enforces a minimal schema.

• Title: string

• Authors: string

• Year: integer or None

• URL: string

Four checks run in order.

1) Normalisation. Strip whitespace. Standardise author delimiters. Fix trivial case issues.

2) Deduplication. Drop exact duplicates using a key of Title and URL. Keep the first occurrence and log the count removed.

3) Validation. Check that years fall within a sensible range. Outframe values become None with a note. 4) Type safety. Confirm column types before export.

The result is a tidy Data Frame. This makes later analysis simple, such as counts by year or author frequency. The module then writes both CSV and Excel versions. The Storage Handler verifies that files exist and that row counts match the in-memory table.

Slide 7 — Demo walkthrough: end-to-end execution

Now I will describe the demo path you will see in the recording. I run a single command to execute the demo script.

• Command: python demo.py

The script reads sample_page.html bundled with the project. This avoids external site changes and keeps the run stable. The controller loads the HTML, calls the Scraper, and passes the content to the parser. The parser yields a list of dicts. Each dict has title, authors, year, and URL. Data Processing converts that list to a Data Frame. It runs normalisation, deduplication, and validation.

The Storage Handler writes outputs to the outputs/ folder. You will see three onscreen elements.

1) Terminal output. The logger prints major steps: “Loaded sample,” “Parsed N records,” “Removed K duplicates,” “Wrote CSV and Excel.”

2) Data preview. The script prints the head of the DataFrame so the viewer can inspect fields.

3) File system view. I switch to the outputs/ folder to show the CSV and Excel files with the timestamp.

For the video, follow these cues.

• Show the terminal at 13:00 when running python demo.py.

• Zoom into the logger lines when the parser reports records and the validator flag any year issues.

• Open the CSV in a viewer and show a few rows.

• Return to the slides.

If live sites are needed for future runs, switch to dynamic mode by enabling the WebDriver. For this assessment I keep the static path to ensure a reliable demo.

Slide 8 — Testing strategy and traceability

Testing covers two levels.

• Unit tests. Check parsing functions against small HTML snippets. Cover missing authors, malformed years, and odd containers.

• Functional tests. Run the full pipeline on the bundled sample. Assert that outputs exist, row counts are positive, and schema is intact.

Each run leaves a trace in the log. The README maps each evidence item to a test or a step in the pipeline. This helps a marker cross check quickly. Screenshots show terminal output, test passes, and exported files. I include these in the Evidence folder.

Slide 9 — Results and evidence

The pipeline generated a clean table of papers from the sample HTML. I plotted a simple count by year to validate the Year column. The Data Frame preview showed normalised author fields and unique rows. All tests passed in the demo run. The outputs folder contained the CSV and Excel with matching row counts. These items appear in the evidence pack, with filenames that match those in the README.

Slide 10 — Design choices, limits, and future work

Key choices.

• Use Python and pandas for clear data handling and strong library support.

• Keep Beautiful Soup as the default for speed and stability. Isolate Selenium for dynamic pages.

• Maintain modular boundaries so new parsers or filters drop in easily. Known limits.

• The demo uses bundled HTML for stability. Real sites vary and need site specific rules.

• Year extraction can be noisy on pages with multiple dates. Future work.

• Add semantic ranking of records. • Improve duplicate detection with fuzzy matching.

• Move towards a full BDI framework for belief updates and failure handling.

Slide 10 — References**:**

Bellifemine, F., Caire, G. and Greenwood, D. (2007) Developing multi-agent systems with JADE. Chichester: John Wiley & Sons.

Booch, G., Rumbaugh, J. and Jacobson, I. (2005) The Unified Modelling Language user guide. 2nd edn. Boston: Addison-Wesley.

Lutz, M. (2013) Learning Python. 5th edn. Sebastopol: O'Reilly Media.

McKinney, W. (2012) Python for data analysis: Data wrangling with pandas, NumPy, and IPython. Sebastopol: O'Reilly Media.

Mitchell, R. (2018) Web scraping with Python: Collecting data from the modern web. 2nd edn. Sebastopol: O'Reilly Media.

Pressman, R. and Maxim, B. (2020) Software engineering: A practitioner's approach. 9th edn. New York: McGraw-Hill.

Van Rossum, G. and Drake, F. (2009) Python 3 reference manual. Scotts Valley: CreateSpace.

Wooldridge, M. (2009) An introduction to multi-agent systems. 2nd edn. Chichester: John Wiley & Sons.

University of Essex Online (2025) Intelligent Agents Development: Individual Project Presentation, Unit 11 assignment brief. Module resource.

Thank you.